GPIO & RTC GPIO

[中文]

GPIO Summary

The ESP32-S3 chip features 45 physical GPIO pins (GPIO0 ~ GPIO21 and GPIO26 ~ GPIO48). Each pin can be used as a general-purpose I/O, or be connected to an internal peripheral signal. Through GPIO matrix, IO MUX, and RTC IO MUX, peripheral input signals can be from any GPIO pin, and peripheral output signals can be routed to any GPIO pin. Together these modules provide highly configurable I/O. For more details, see *ESP32-S3 Technical Reference Manual > IO MUX and GPIO Matrix (GPIO, IO_MUX)* [PDF].

The table below provides more information on pin usage, and please note the comments in the table for GPIOs with restrictions.

GPIO	Analog Function	RTC GPIO	Comment
GPIO0		RTC_GPIO0	Strapping pin
GPIO1	ADC1_CH0	RTC_GPIO1	
GPIO2	ADC1_CH1	RTC_GPIO2	
GPIO3	ADC1_CH2	RTC_GPIO3	Strapping pin
GPIO4	ADC1_CH3	RTC_GPIO4	
GPIO5	ADC1_CH4	RTC_GPIO5	
GPIO6	ADC1_CH5	RTC_GPIO6	
GPIO7	ADC1_CH6	RTC_GPIO7	
GPIO8	ADC1_CH7	RTC_GPIO8	
GPIO9	ADC1_CH8	RTC_GPIO9	
GPIO10	ADC1_CH9	RTC_GPIO10	
GPIO11	ADC2_CH0	RTC_GPIO11	
GPIO12	ADC2_CH1	RTC_GPIO12	
GPIO13	ADC2_CH2	RTC_GPIO13	
GPIO14	ADC2_CH3	RTC_GPIO14	

GPIO	Analog Function	RTC GPIO	Comment
GPIO15	ADC2_CH4	RTC_GPIO15	
GPIO16	ADC2_CH5	RTC_GPIO16	
GPIO17	ADC2_CH6	RTC_GPIO17	
GPIO18	ADC2_CH7	RTC_GPIO18	
GPIO19	ADC2_CH8	RTC_GPIO19	USB-JTAG
GPIO20	ADC2_CH9	RTC_GPIO20	USB-JTAG
GPIO21		RTC_GPIO21	
GPIO26			SPI0/1
GPIO27			SPI0/1
GPIO28			SPI0/1
GPIO29			SPI0/1
GPIO30			SPI0/1
GPIO31			SPI0/1
GPIO32			SPI0/1
GPIO33			SPI0/1
GPIO34			SPI0/1
GPIO35			SPI0/1
GPIO36			SPI0/1
GPIO37			SPI0/1
GPIO38			
GPIO39			
GPIO40			
GPIO41			
GPIO42			
GPIO43			
GPIO44			
GPIO45			Strapping pin
GPIO46			Strapping pin
GPIO47			
GPIO48			

Note

- Strapping pin: GPIO0, GPIO3, GPIO45 and GPIO46 are strapping pins. For more information, please refer to ESP32-S3 datasheet.
- SPI0/1: GPIO26 ~ GPIO32 are usually used for SPI flash and PSRAM and not recommended for other uses. When using Octal flash or Octal PSRAM or both, GPIO33 ~ GPIO37 are connected to SPIIO4 ~ SPIIO7 and SPIDQS. Therefore, on boards embedded with ESP32-S3R8 / ESP32-S3R8V chip, GPIO33 ~ GPIO37 are also not recommended for other uses.
- USB-JTAG: GPIO19 and GPIO20 are used by USB-JTAG by default. If they are reconfigured to operate as normal GPIOs, USB-JTAG functionality will be disabled.

There is also separate "RTC GPIO" support, which functions when GPIOs are routed to the "RTC" low-power and analog subsystem. These pin functions can be used when:

- In Deep-sleep mode
- The Ultra Low Power FSM co-processor is running
- The Ultra Low Power RISC-V co-processor is running
- Analog functions such as ADC/DAC/etc are in use

IO Configuration

An IO can be used in two ways:

- As a simple GPIO input to read the level on the pin, or as a simple GPIO output to output the desired level on the pin.
- As a peripheral signal input/output.

IDF peripheral drivers always take care of the necessary IO configurations that need to be applied onto the pins, so that they can be used as the peripheral signal inputs or outputs. This means the users usually only need to be responsible for configuring the IOs as simple inputs or outputs. `gpio_config()` is an all-in-one API that can be used to configure the I/O mode, internal pull-up/pull-down resistors, etc. for pins, including the ones reused by the USB PHY.

In some applications, an IO pin can serve dual purposes. For example, the IO, which outputs a LEDC PWM signal, can also act as a GPIO input to generate interrupts or GPIO ETM events. Careful handling on the configuration step is necessary for such dual use of IO pins cases. `gpio_config()` is an API that overwrites all the current configurations, so it must be called to set the pin mode to `gpio_mode_t::GPIO_MODE_INPUT` before calling the LEDC driver API which connects the output signal to the pin. As an alternative, if no other configuration is needed other than making the pin input enabled, `gpio_input_enable()` can be the one to call at any time to achieve the same purpose.

Check Current Configuration of IOs

GPIO driver offers a dump function `gpio_dump_io_configuration()` to show the current configurations of IOs, such as pull-up/pull-down, input/output enable, pin mapping, etc. Below is an example of how to dump the configuration of GPIO4, GPIO18, and GPIO26:

```
gpio_dump_io_configuration(stdout, (1ULL << 4) | (1ULL << 18) | (1ULL << 26));
```

The dump will be like this:

```
================IO DUMP Start================
IO[4] -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigIn ID: (simple GPIO input)
  SleepSelEn: 1

IO[18] -
  Pullup: 0, Pulldown: 0, DriveCap: 2
  InputEn: 0, OutputEn: 1, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigOut ID: 256 (simple GPIO output)
  SleepSelEn: 1

IO[26] **RESERVED** -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 0 (IOMUX)
  SleepSelEn: 1

================IO DUMP End================
```

In addition, if you would like to dump the configurations of all IOs, you can use:

```
gpio_dump_io_configuration(stdout, SOC_GPIO_VALID_GPIO_MASK);
```

If an IO pin is routed to a peripheral signal through the GPIO matrix, the signal ID printed in the dump information is defined in the soc/esp32s3/include/soc/gpio_sig_map.h header file. The word `**RESERVED**` indicates the IO is occupied by either SPI flash or PSRAM. It is strongly not recommended to reconfigure them for other application purposes.

Do not rely on the default configurations values in the Technical Reference Manual, because it may be changed in the bootloader or application startup code before app_main.

GPIO Glitch Filter

The ESP32-S3 chip features hardware filters to remove unwanted glitch pulses from the input GPIO, which can help reduce false triggering of the interrupt and prevent a noise being routed to the peripheral side.

Each GPIO can be configured with a glitch filter, which can be used to filter out pulses shorter than **two** sample clock cycles. The duration of the filter is not configurable. The sample clock is the clock source of the IO_MUX. In the driver, we call this kind of filter as `pin glitch filter`. You can create the filter handle by calling `gpio_new_pin_glitch_filter()`. All the configurations for a pin glitch filter are listed in the `gpio_pin_glitch_filter_config_t` structure.

- `gpio_pin_glitch_filter_config_t::gpio_num` sets the GPIO number to enable the glitch filter.

The glitch filter is disabled by default, and can be enabled by calling `gpio_glitch_filter_enable()`. To recycle the filter, you can call `gpio_del_glitch_filter()`. Please note, before deleting the filter, you should disable it first by calling `gpio_glitch_filter_disable()`.

Application Example

- peripherals/gpio/generic_gpio demonstrates how to configure GPIO to generate pulses and use it with interruption.

API Reference - Normal GPIO

Header File

- components/esp_driver_gpio/include/driver/gpio.h
- This header file can be included with:

```
#include "driver/gpio.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

esp_err_t **gpio_config**(*const* gpio_config_t *pGPIOConfig)

GPIO common configuration.

```
Configure GPIO's Mode,pull-up,PullDown,IntrType
```

Note

This function always overwrite all the current IO configurations

Parameters: **pGPIOConfig** -- Pointer to GPIO configure struct

Returns:

- ESP_OK success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_reset_pin**(gpio_num_t gpio_num)

Reset a GPIO to a certain state (select gpio function, enable pullup and disable input and output).

Parameters: **gpio_num** -- GPIO number.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_set_intr_type**(gpio_num_t gpio_num, gpio_int_type_t intr_type)

GPIO set interrupt trigger type.

Parameters:

- **gpio_num** -- GPIO number. If you want to set the trigger type of e.g. of GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **intr_type** -- Interrupt type, select from gpio_int_type_t

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_intr_enable**(gpio_num_t gpio_num)

Enable GPIO module interrupt signal.

Note

ESP32: Please do not use the interrupt of GPIO36 and GPIO39 when using ADC or Wi-Fi and Bluetooth with sleep mode enabled. Please refer to the comments of `adc1_get_raw`. Please refer to GPIO-3.11 of ESP32 ECO and Workarounds for Bugs for the description of this issue.

Parameters: **gpio_num** -- GPIO number. If you want to enable an interrupt on e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_intr_disable**(gpio_num_t gpio_num)

Disable GPIO module interrupt signal.

Note

This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters: **gpio_num** -- GPIO number. If you want to disable the interrupt of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns:

- ESP_OK success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_set_level**(gpio_num_t gpio_num, uint32_t level)

GPIO set output level.

Note

This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters:
- **gpio_num** -- GPIO number. If you want to set the output level of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **level** -- Output level. 0: low ; 1: high

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

---

int **gpio_get_level**(gpio_num_t gpio_num)

GPIO get input level.

Warning

If the pad is not configured for input (or input and output) the returned value is always 0.

Parameters: **gpio_num** -- GPIO number. If you want to get the logic level of e.g. pin GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns:
- 0 the GPIO input level is 0
- 1 the GPIO input level is 1

---

esp_err_t **gpio_set_direction**(gpio_num_t gpio_num, gpio_mode_t mode)

GPIO set direction.

Configure GPIO mode,such as output_only,input_only,output_and_input

Note

This function always overwrite all the current modes that have applied on the IO pin

Parameters:
- **gpio_num** -- Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** -- GPIO direction

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

---

esp_err_t **gpio_input_enable**(gpio_num_t gpio_num)

Enable input for an IO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

---

esp_err_t **gpio_set_pull_mode**(gpio_num_t gpio_num, gpio_pull_mode_t pull)

Configure GPIO internal pull-up/pull-down resistors.

Note

This function always overwrite the current pull-up/pull-down configurations

Note

ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

Parameters:

- **gpio_num** -- GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **pull** -- GPIO pull up/down mode.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG : Parameter error

---

esp_err_t **gpio_wakeup_enable**(gpio_num_t gpio_num, gpio_int_type_t intr_type)

Enable GPIO wake-up function.

Parameters:

- **gpio_num** -- GPIO number.
- **intr_type** -- GPIO wake-up type. Only GPIO_INTR_LOW_LEVEL or GPIO_INTR_HIGH_LEVEL can be used.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_wakeup_disable**(gpio_num_t gpio_num)

Disable GPIO wake-up function.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_isr_register**(void (*fn)(void*), void *arg, int intr_alloc_flags, gpio_isr_handle_t *handle)

Register GPIO interrupt handler, the handler is an ISR. The handler will be attached to the same CPU core that this function is running on.

This ISR function is called whenever any GPIO interrupt occurs. See the alternative gpio_install_isr_service() and gpio_isr_handler_add() API in order to have the driver support per-GPIO ISRs.

To disable or remove the ISR, pass the returned handle to the interrupt allocation functions.

Parameters:
- **fn** -- Interrupt handler function.
- **arg** -- Parameter for handler function
- **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.
- **handle** -- Pointer to return handle. If non-NULL, a handle for the interrupt will be returned here.

Returns:
- ESP_OK Success ;
- ESP_ERR_INVALID_ARG GPIO error
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags

---

esp_err_t **gpio_pullup_en**(gpio_num_t gpio_num)

Enable pull-up on GPIO.

Parameters: **gpio_num** -- GPIO number

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_pullup_dis**(gpio_num_t gpio_num)

Disable pull-up on GPIO.

Parameters: **gpio_num** -- GPIO number

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_pulldown_en**(gpio_num_t gpio_num)

Enable pull-down on GPIO.

Parameters: **gpio_num** -- GPIO number

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_pulldown_dis**(gpio_num_t gpio_num)

Disable pull-down on GPIO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_output_enable**(gpio_num_t gpio_num)

Enable output for an IO (as a simple GPIO output)

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

esp_err_t **gpio_output_disable**(gpio_num_t gpio_num)

Disable output for an IO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

esp_err_t **gpio_od_enable**(gpio_num_t gpio_num)

Enable open-drain for an IO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

esp_err_t **gpio_od_disable**(gpio_num_t gpio_num)

Disable open-drain for an IO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

esp_err_t **gpio_install_isr_service**(int intr_alloc_flags)

Install the GPIO driver's ETS_GPIO_INTR_SOURCE ISR handler service, which allows per-pin

GPIO interrupt handlers.

This function is incompatible with gpio_isr_register() - if that function is used, a single global ISR is registered for all GPIO interrupts. If this function is used, the ISR service provides a global GPIO ISR and individual pin handlers are registered via the gpio_isr_handler_add() function.

Parameters: **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.

Returns:

- ESP_OK Success
- ESP_ERR_NO_MEM No memory to install this service
- ESP_ERR_INVALID_STATE ISR service already installed.
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags
- ESP_ERR_INVALID_ARG GPIO error

---

esp_err_t **gpio_uninstall_isr_service**(void)

Uninstall the driver's GPIO ISR service, freeing related resources.

Returns:

- ESP_OK Success

---

esp_err_t **gpio_isr_handler_add**(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args)

Add ISR handler for the corresponding GPIO pin.

Call this function after using gpio_install_isr_service() to install the driver's GPIO ISR handler service.

The pin ISR handlers no longer need to be declared with IRAM_ATTR, unless you pass the ESP_INTR_FLAG_IRAM flag when allocating the ISR in gpio_install_isr_service().

This ISR handler will be called from an ISR. So there is a stack size limit (configurable as "ISR stack size" in menuconfig). This limit is smaller compared to a global GPIO interrupt handler due to the additional level of indirection.

Parameters:

- **gpio_num** -- GPIO number
- **isr_handler** -- ISR handler function for the corresponding GPIO number.
- **args** -- parameter for ISR handler.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_isr_handler_remove**(gpio_num_t gpio_num)

Remove ISR handler for the corresponding GPIO pin.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_set_drive_capability**(gpio_num_t gpio_num, gpio_drive_cap_t strength)

Set GPIO pad drive capability.

Parameters:

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_get_drive_capability**(gpio_num_t gpio_num, gpio_drive_cap_t *strength)

Get GPIO pad drive capability.

Parameters:

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Pointer to accept drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_hold_en**(gpio_num_t gpio_num)

Enable gpio pad hold function.

When a GPIO is set to hold, its state is latched at that moment and will not change when the internal signal or the IO MUX/GPIO configuration is modified (including input enable, output enable, output value, function, and drive strength values). This function can be used to retain the state of GPIOs when the power domain of where GPIO/IOMUX belongs to becomes off. For example, chip or system is reset (e.g. watchdog time-out, Deep-sleep events are triggered), or peripheral power-down in Light-sleep.

This function works in both input and output modes, and only applicable to output-capable GPIOs. If this function is enabled: in output mode: the output level of the GPIO will be locked and can not be changed. in input mode: the input read value can still reflect the changes of the input signal.

Power down or call `gpio_hold_dis` will disable this function.

Please be aware that,

a. USB pads cannot hold at low level after waking up from Deep-sleep. The USB related

registers are reset, so the USB pull-up is back.

b. For ESP32-P4 rev < 3.0, the states of IOs can not be hold after waking up from Deep-sleep.

c. For ESP32/S2/C3/S3/C2, this function cannot be used to hold the state of a digital GPIO during Deep-sleep. Even if this function is enabled, the digital GPIO will be reset to its default state when the chip wakes up from Deep-sleep. If you want to hold the state of a digital GPIO during Deep-sleep, please call `gpio_deep_sleep_hold_en`.

Parameters: **gpio_num** -- GPIO number, only support output-capable GPIOs

Returns:

- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

---

esp_err_t **gpio_hold_dis**(gpio_num_t gpio_num)

Disable gpio pad hold function.

When the chip is woken up from peripheral power-down sleep, the gpio will be set to the default mode, so, the gpio will output the default level if this function is called. If you don't want the level changes, the gpio should be configured to a known state before this function is called. e.g. If you hold gpio18 high during Deep-sleep, after the chip is woken up and `gpio_hold_dis` is called, gpio18 will output low level(because gpio18 is input mode by default). If you don't want this behavior, you should configure gpio18 as output mode and set it to high level before calling `gpio_hold_dis`.

Parameters: **gpio_num** -- GPIO number, only support output-capable GPIOs

Returns:

- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

---

void **gpio_deep_sleep_hold_en**(void)

Enable all digital gpio pads hold function during Deep-sleep.

Enabling this feature makes all digital gpio pads be at the holding state during Deep-sleep. The state of each pad holds is its active configuration (not pad's sleep configuration!).

Note:

a. For digital IO, this API takes effect only if the corresponding digital IO pad hold function has been enabled. You can enable the GPIO pad hold function by calling `gpio_hold_en`. has been enabled. You can call `gpio_hold_en` to enable the gpio pad hold function.

b. Though this API targets all digital IOs, the pad hold feature only works when the chip is in Deep-sleep mode. When the chip is in active mode, the digital GPIO state can be changed freely even if you have called this function, except for IOs that are already held by `gpio_hold_en`.

After this API is being called, the digital gpio Deep-sleep hold feature will work during every

sleep process. You should call `gpio_deep_sleep_hold_dis` to disable this feature.

void **gpio_deep_sleep_hold_dis**(void)

Disable all digital gpio pads hold function during Deep-sleep.

esp_err_t **gpio_force_hold_all**(void)

SOC_GPIO_SUPPORT_HOLD_SINGLE_IO_IN_DSLP.

Force hold all digital and rtc gpio pads.

GPIO force hold, no matter the chip in active mode or sleep modes.

This function will immediately cause all pads to latch the current values of input enable, output enable, output value, function, and drive strength values.

❗ Warning

a. This function will hold flash and UART pins as well. Therefore, this function, and all code run afterwards (till calling `gpio_force_unhold_all` to disable this feature), MUST be placed in internal RAM as holding the flash pins will halt SPI flash operation, and holding the UART pins will halt any UART logging.
b. The hold state of all pads will be cancelled during ROM boot, so it is not recommended to use this API to hold the pads state during deepsleep and reset.

esp_err_t **gpio_force_unhold_all**(void)

Unhold all digital and rtc gpio pads.

❗ Note

The global hold signal and the hold signal of each IO act on the PAD through 'or' logic, so if a pad has already been configured to hold by `gpio_hold_en`, this API can't release its hold state.

esp_err_t **gpio_sleep_sel_en**(gpio_num_t gpio_num)

Enable SLP_SEL to change GPIO status automantically in lightsleep.

Parameters: **gpio_num** -- GPIO number of the pad.

Returns:

- ESP_OK Success

esp_err_t **gpio_sleep_sel_dis**(gpio_num_t gpio_num)

Disable SLP_SEL to change GPIO status automantically in lightsleep.

Parameters: **gpio_num** -- GPIO number of the pad.

Returns:

- ESP_OK Success

---

esp_err_t **gpio_sleep_set_direction**(gpio_num_t gpio_num, gpio_mode_t mode)

GPIO set direction at sleep.

Configure GPIO direction,such as output_only,input_only,output_and_input

Parameters:

- **gpio_num** -- Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** -- GPIO direction

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

---

esp_err_t **gpio_sleep_set_pull_mode**(gpio_num_t gpio_num, gpio_pull_mode_t pull)

Configure GPIO pull-up/pull-down resistors at sleep.

Note

ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

Parameters:

- **gpio_num** -- GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **pull** -- GPIO pull up/down mode.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG : Parameter error

---

esp_err_t **gpio_wakeup_enable_on_hp_periph_powerdown_sleep**(gpio_num_t gpio_num, gpio_int_type_t intr_type)

Enable GPIO wake-up function on peripheral powerdowned sleep (including deepsleep and peripheral powerdowned lightsleep).

Note

Called by the SDK. User shouldn't call this directly in the APP.

Parameters:
- **gpio_num** -- GPIO number.
- **intr_type** -- GPIO wake-up type.
  - Always supported: GPIO_INTR_LOW_LEVEL, GPIO_INTR_HIGH_LEVEL
  - If SOC_RTC_GPIO_EDGE_WAKEUP_SUPPORTED: GPIO_INTR_POSEDGE, GPIO_INTR_NEGEDGE, GPIO_INTR_ANYEDGE

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_wakeup_disable_on_hp_periph_powerdown_sleep**(gpio_num_t gpio_num)

Disable GPIO peripheral powerdowned sleep (including deepsleep and peripheral powerdowned lightsleep) wake-up function.

Parameters: **gpio_num** -- GPIO number

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_dump_io_configuration**(FILE *out_stream, uint64_t io_bit_mask)

Dump IO configuration information to console.

Parameters:
- **out_stream** -- IO stream (e.g. stdout)
- **io_bit_mask** -- IO pin bit mask, each bit maps to an IO

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **gpio_get_io_config**(gpio_num_t gpio_num, gpio_io_config_t *out_io_config)

Get the configuration for an IO.

Parameters:
- **gpio_num** -- GPIO number
- **out_io_config** -- **[out]** Pointer to the structure that saves the specific IO configuration

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

Structures

---

struct **gpio_config_t**

Configuration parameters of GPIO pad for gpio_config function.

Public Members

> **uint64_t pin_bit_mask**
>
> GPIO pin: set with bit mask, each bit maps to a GPIO

> gpio_mode_t **mode**
>
> GPIO mode: set input/output mode

> gpio_pullup_t **pull_up_en**
>
> GPIO pull-up

> gpio_pulldown_t **pull_down_en**
>
> GPIO pull-down

> gpio_int_type_t **intr_type**
>
> GPIO interrupt type

Macros

GPIO_IS_HP_PERIPH_PD_WAKEUP_VALID_IO(gpio_num)

Type Definitions

typedef intr_handle_t **gpio_isr_handle_t**

typedef void (***gpio_isr_t**)(void *arg)

GPIO interrupt handler.

Param arg: User registered data

Header File

- components/esp_hal_gpio/include/hal/gpio_types.h
- This header file can be included with:

```
#include "hal/gpio_types.h"
```

- This header file is a part of the API provided by the `esp_hal_gpio` component. To declare that

your component depends on `esp_hal_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_hal_gpio
```

or

```
PRIV_REQUIRES esp_hal_gpio
```

Structures

struct `gpio_io_config_t`

Structure that contains the configuration of an IO.

Public Members

gpio_drive_cap_t **drv**

Value of drive strength

uint32_t fun_sel

Value of IOMUX function selection

uint32_t sig_out

Index of the outputting peripheral signal

uint32_t pu

Status of pull-up enabled or not

uint32_t pd

Status of pull-down enabled or not

uint32_t ie

Status of input enabled or not

uint32_t oe

Status of output enabled or not

uint32_t oe_ctrl_by_periph

True if use output enable signal from peripheral, otherwise False

uint32_t oe_inv

Whether the output enable signal is inversed or not

uint32_t od

Status of open-drain enabled or not

uint32_t slp_sel

Status of pin sleep mode enabled or not

Macros

GPIO_PIN_COUNT

GPIO_IS_VALID_GPIO(gpio_num)

Check whether it is a valid GPIO number.

GPIO_IS_VALID_OUTPUT_GPIO(gpio_num)

Check whether it can be a valid GPIO number of output mode.

GPIO_IS_VALID_DIGITAL_IO_PAD(gpio_num)

Check whether it can be a valid digital I/O pad.

GPIO_PIN_REG_0

GPIO_PIN_REG_1

GPIO_PIN_REG_2

GPIO_PIN_REG_3

GPIO_PIN_REG_4

GPIO_PIN_REG_5

GPIO_PIN_REG_6

GPIO_PIN_REG_7

GPIO_PIN_REG_8

GPIO_PIN_REG_9

GPIO_PIN_REG_10

GPIO_PIN_REG_11

GPIO_PIN_REG_12

GPIO_PIN_REG_13

GPIO_PIN_REG_14

GPIO_PIN_REG_15

GPIO_PIN_REG_16

GPIO_PIN_REG_17

GPIO_PIN_REG_18

GPIO_PIN_REG_19

GPIO_PIN_REG_20

GPIO_PIN_REG_21

GPIO_PIN_REG_22

GPIO_PIN_REG_23

GPIO_PIN_REG_24

GPIO_PIN_REG_25

GPIO_PIN_REG_26

GPIO_PIN_REG_27

GPIO_PIN_REG_28

GPIO_PIN_REG_29

GPIO_PIN_REG_30

GPIO_PIN_REG_31

GPIO_PIN_REG_32

GPIO_PIN_REG_33

GPIO_PIN_REG_34

GPIO_PIN_REG_35

GPIO_PIN_REG_36

GPIO_PIN_REG_37

GPIO_PIN_REG_38

GPIO_PIN_REG_39

GPIO_PIN_REG_40

GPIO_PIN_REG_41

GPIO_PIN_REG_42

GPIO_PIN_REG_43

GPIO_PIN_REG_44

GPIO_PIN_REG_45

GPIO_PIN_REG_46

GPIO_PIN_REG_47

GPIO_PIN_REG_48

GPIO_PIN_REG_49

GPIO_PIN_REG_50

GPIO_PIN_REG_51

GPIO_PIN_REG_52

GPIO_PIN_REG_53

GPIO_PIN_REG_54

Enumerations

enum **gpio_port_t**

Values:

***enumerator* GPIO_PORT_0**

***enumerator* GPIO_PORT_MAX**

enum **gpio_int_type_t**

Values:

***enumerator* GPIO_INTR_DISABLE**

Disable GPIO interrupt

***enumerator* GPIO_INTR_POSEDGE**

GPIO interrupt type : rising edge

enumerator **GPIO_INTR_NEGEDGE**

GPIO interrupt type : falling edge

enumerator **GPIO_INTR_ANYEDGE**

GPIO interrupt type : both rising and falling edge

enumerator **GPIO_INTR_LOW_LEVEL**

GPIO interrupt type : input low level trigger

enumerator **GPIO_INTR_HIGH_LEVEL**

GPIO interrupt type : input high level trigger

enumerator **GPIO_INTR_MAX**

enum **gpio_mode_t**

Values:

enumerator **GPIO_MODE_DISABLE**

GPIO mode : disable input and output

enumerator **GPIO_MODE_INPUT**

GPIO mode : input only

enumerator **GPIO_MODE_OUTPUT**

GPIO mode : output only mode

enumerator **GPIO_MODE_OUTPUT_OD**

GPIO mode : output only with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT_OD**

GPIO mode : output and input with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT**

GPIO mode : output and input mode

enum **gpio_pullup_t**

Values:

enumerator **GPIO_PULLUP_DISABLE**

Disable GPIO pull-up resistor

enumerator **GPIO_PULLUP_ENABLE**

Enable GPIO pull-up resistor

enum **gpio_pulldown_t**

Values:

enumerator **GPIO_PULLDOWN_DISABLE**

Disable GPIO pull-down resistor

enumerator **GPIO_PULLDOWN_ENABLE**

Enable GPIO pull-down resistor

enum **gpio_pull_mode_t**

Values:

enumerator **GPIO_PULLUP_ONLY**

Pad pull up

enumerator **GPIO_PULLDOWN_ONLY**

Pad pull down

enumerator **GPIO_PULLUP_PULLDOWN**

Pad pull up + pull down

enumerator **GPIO_FLOATING**

Pad floating

enum **gpio_drive_cap_t**

Values:

enumerator **GPIO_DRIVE_CAP_0**

Pad drive capability: weak

enumerator **GPIO_DRIVE_CAP_1**

Pad drive capability: stronger

enumerator **GPIO_DRIVE_CAP_2**

Pad drive capability: medium

enumerator **GPIO_DRIVE_CAP_DEFAULT**

Pad drive capability: medium

enumerator **GPIO_DRIVE_CAP_3**

Pad drive capability: strongest

enumerator **GPIO_DRIVE_CAP_MAX**

API Reference - RTC GPIO

Header File

- components/esp_driver_gpio/include/driver/rtc_io.h
- This header file can be included with:

```
#include "driver/rtc_io.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

bool **rtc_gpio_is_valid_gpio**(gpio_num_t gpio_num)

Determine if the specified IO is a valid RTC GPIO.

Parameters: **gpio_num** -- GPIO number

Returns: true if the IO is valid for RTC GPIO use. false otherwise.

int **rtc_io_number_get**(gpio_num_t gpio_num)

Get RTC IO index number by GPIO number.

Parameters: **gpio_num** -- GPIO number

Returns: >=0: Index of RTC IO. -1 : The IO is not an RTC IO.

esp_err_t **rtc_gpio_init**(gpio_num_t gpio_num)

Init an IO to be an RTC GPIO, route to RTC IO MUX.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

esp_err_t **rtc_gpio_deinit**(gpio_num_t gpio_num)

Deinit an IO as an RTC GPIO, route back to IO MUX.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

uint32_t **rtc_gpio_get_level**(gpio_num_t gpio_num)

Get the RTC IO input level.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- 1 High level
- 0 Low level
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

esp_err_t **rtc_gpio_set_level**(gpio_num_t gpio_num, uint32_t level)

Set the RTC IO output level.

Parameters:

- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **level** -- output level

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

esp_err_t **rtc_gpio_set_direction**(gpio_num_t gpio_num, rtc_gpio_mode_t mode)

RTC GPIO set direction.

Configure RTC GPIO direction, such as output only, input only, output and input.

Parameters:
- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **mode** -- RTC GPIO direction

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_set_direction_in_sleep**(gpio_num_t gpio_num, rtc_gpio_mode_t mode)

RTC GPIO set direction in deep sleep mode or disable sleep status (default). In some application scenarios, IO needs to have another states during deep sleep.

NOTE: ESP32 supports INPUT_ONLY mode. The rest targets support INPUT_ONLY, OUTPUT_ONLY, INPUT_OUTPUT mode.

Parameters:
- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **mode** -- RTC GPIO direction

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_pullup_en**(gpio_num_t gpio_num)

RTC GPIO pullup enable.

This function only works for RTC IOs. In general, call gpio_pullup_en, which will work both for normal GPIOs and RTC IOs.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_pulldown_en**(gpio_num_t gpio_num)

RTC GPIO pulldown enable.

This function only works for RTC IOs. In general, call gpio_pulldown_en, which will work both for normal GPIOs and RTC IOs.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:
- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_pullup_dis**(gpio_num_t gpio_num)

RTC GPIO pullup disable.

This function only works for RTC IOs. In general, call gpio_pullup_dis, which will work both for normal GPIOs and RTC IOs.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_pulldown_dis**(gpio_num_t gpio_num)

RTC GPIO pulldown disable.

This function only works for RTC IOs. In general, call gpio_pulldown_dis, which will work both for normal GPIOs and RTC IOs.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_set_drive_capability**(gpio_num_t gpio_num, gpio_drive_cap_t strength)

Set RTC GPIO pad drive capability.

Parameters:

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **rtc_gpio_get_drive_capability**(gpio_num_t gpio_num, gpio_drive_cap_t *strength)

Get RTC GPIO pad drive capability.

Parameters:

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Pointer to accept drive capability of the pad

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **rtc_gpio_iomux_func_sel**(gpio_num_t gpio_num, int func)

Select a RTC IOMUX function for the RTC IO.

Parameters:

- **gpio_num** -- GPIO number
- **func** -- Function to assign to the pin

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

---

esp_err_t **rtc_gpio_iomux_input**(gpio_num_t gpio_num, int func, uint32_t signal_idx)

Set pad input to an LP peripheral signal through the LP(RTC) IOMUX.

Parameters:

- **gpio_num** -- GPIO number
- **func** -- The index number of the LP(RTC) IOMUX function to be selected for the pin
- **signal_idx** -- Peripheral signal index to input. One of the `*_IN_IDX` signals in `soc/lp_gpio_sig_map.h`.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

---

esp_err_t **rtc_gpio_iomux_output**(gpio_num_t gpio_num, int func)

Set LP peripheral output to an RTC IO pad through the LP(RTC) IOMUX.

Parameters:

- **gpio_num** -- GPIO number
- **func** -- The index number of the LP(RTC) IOMUX function to be selected for the pin

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

---

esp_err_t **rtc_gpio_hold_en**(gpio_num_t gpio_num)

Enable hold function on an RTC IO pad.

Enabling HOLD function will cause the pad to latch current values of input enable, output enable, output value, function, drive strength values. This function is useful when going into light or deep sleep mode to prevent the pin configuration from changing.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_hold_dis**(gpio_num_t gpio_num)

Disable hold function on an RTC IO pad.

Disabling hold function will allow the pad receive the values of input enable, output enable, output value, function, drive strength from RTC_IO peripheral.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_force_hold_en_all**(void)

Enable force hold signal for all RTC IOs.

Each RTC pad has a "force hold" input signal from the RTC controller. If this signal is set, pad latches current values of input enable, function, output enable, and other signals which come from the RTC mux. Force hold signal is enabled before going into deep sleep for pins which are used for EXT1 wakeup.

---

esp_err_t **rtc_gpio_force_hold_dis_all**(void)

Disable force hold signal for all RTC IOs.

---

esp_err_t **rtc_gpio_isolate**(gpio_num_t gpio_num)

Helper function to disconnect internal circuits from an RTC IO This function disables input, output, pullup, pulldown, and enables hold feature for an RTC IO. Use this function if an RTC IO needs to be disconnected from internal circuits in deep sleep, to minimize leakage current.

In particular, for ESP32-WROVER module, call rtc_gpio_isolate(GPIO_NUM_12) before entering deep sleep, to reduce deep sleep current.

Parameters: **gpio_num** -- GPIO number (e.g. GPIO_NUM_12).

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

---

esp_err_t **rtc_gpio_wakeup_enable**(gpio_num_t gpio_num, gpio_int_type_t intr_type)

Enable wakeup from sleep mode using specific GPIO.

Parameters:

- **gpio_num** -- GPIO number
- **intr_type** -- Wakeup trigger type:
  - Always supported: GPIO_INTR_HIGH_LEVEL, GPIO_INTR_LOW_LEVEL
  - If SOC_RTC_GPIO_EDGE_WAKEUP_SUPPORTED: GPIO_INTR_POSEDGE, GPIO_INTR_NEGEDGE, GPIO_INTR_ANYEDGE

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO, or intr_type is not supported by the target.

esp_err_t **rtc_gpio_wakeup_disable**(gpio_num_t gpio_num)

Disable wakeup from sleep mode using specific GPIO.

Parameters: **gpio_num** -- GPIO number

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG The IO is not an RTC IO

Macros

RTC_GPIO_IS_VALID_GPIO(gpio_num)

Header File

- components/esp_driver_gpio/include/driver/lp_io.h
- This header file can be included with:

```
#include "driver/lp_io.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Header File

- components/esp_hal_gpio/include/hal/rtc_io_types.h
- This header file can be included with:

```
#include "hal/rtc_io_types.h"
```

- This header file is a part of the API provided by the `esp_hal_gpio` component. To declare that your component depends on `esp_hal_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_hal_gpio
```

or

```
PRIV_REQUIRES esp_hal_gpio
```

Enumerations

enum **rtc_gpio_mode_t**

RTCIO output/input mode type.

Values:

> ***enumerator*** **RTC_GPIO_MODE_INPUT_ONLY**
>
> Pad input

> ***enumerator*** **RTC_GPIO_MODE_OUTPUT_ONLY**
>
> Pad output

> ***enumerator*** **RTC_GPIO_MODE_INPUT_OUTPUT**
>
> Pad input + output

> ***enumerator*** **RTC_GPIO_MODE_DISABLED**
>
> Pad (output + input) disable

> ***enumerator*** **RTC_GPIO_MODE_OUTPUT_OD**
>
> Pad open-drain output

> ***enumerator*** **RTC_GPIO_MODE_INPUT_OUTPUT_OD**
>
> Pad input + open-drain output

API Reference - GPIO Glitch Filter

Header File

- components/esp_driver_gpio/include/driver/gpio_filter.h
- This header file can be included with:

```
#include "driver/gpio_filter.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

esp_err_t **gpio_new_pin_glitch_filter**(*const* gpio_pin_glitch_filter_config_t *config, gpio_glitch_filter_handle_t *ret_filter)

Create a pin glitch filter.

Note

Pin glitch filter parameters are fixed, pulses shorter than two sample clocks (IO-MUX's source clock) will be filtered out. It's independent with "flex" glitch filter. See also `gpio_new_flex_glitch_filter`.

Note

The created filter handle can later be deleted by `gpio_del_glitch_filter`.

Parameters:
- **config** -- **[in]** Glitch filter configuration
- **ret_filter** -- **[out]** Returned glitch filter handle

Returns:
- ESP_OK: Create a pin glitch filter successfully
- ESP_ERR_INVALID_ARG: Create a pin glitch filter failed because of invalid arguments (e.g. wrong GPIO number)
- ESP_ERR_NO_MEM: Create a pin glitch filter failed because of out of memory
- ESP_FAIL: Create a pin glitch filter failed because of other error

esp_err_t **gpio_new_flex_glitch_filter**(*const* gpio_flex_glitch_filter_config_t *config,

gpio_glitch_filter_handle_t *ret_filter)

Allocate a flex glitch filter.

Note

"flex" means the filter parameters (window, threshold) are adjustable. It's independent with pin glitch filter. See also `gpio_new_pin_glitch_filter`.

Note

The created filter handle can later be deleted by `gpio_del_glitch_filter`.

Parameters:
- **config** -- **[in]** Glitch filter configuration
- **ret_filter** -- **[out]** Returned glitch filter handle

Returns:
- ESP_OK: Allocate a flex glitch filter successfully
- ESP_ERR_INVALID_ARG: Allocate a flex glitch filter failed because of invalid arguments (e.g. wrong GPIO number, filter parameters out of range)
- ESP_ERR_NO_MEM: Allocate a flex glitch filter failed because of out of memory
- ESP_ERR_NOT_FOUND: Allocate a flex glitch filter failed because the underlying hardware resources are used up
- ESP_FAIL: Allocate a flex glitch filter failed because of other error

---

esp_err_t **gpio_del_glitch_filter**(gpio_glitch_filter_handle_t filter)

Delete a glitch filter.

Parameters: **filter** -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns:
- ESP_OK: Delete glitch filter successfully
- ESP_ERR_INVALID_ARG: Delete glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Delete glitch filter failed because the glitch filter is still in working
- ESP_FAIL: Delete glitch filter failed because of other error

---

esp_err_t **gpio_glitch_filter_enable**(gpio_glitch_filter_handle_t filter)

Enable a glitch filter.

Parameters: **filter** -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns:

- ESP_OK: Enable glitch filter successfully
- ESP_ERR_INVALID_ARG: Enable glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Enable glitch filter failed because the glitch filter is already enabled
- ESP_FAIL: Enable glitch filter failed because of other error

---

esp_err_t **gpio_glitch_filter_disable**(gpio_glitch_filter_handle_t filter)

Disable a glitch filter.

Parameters: **filter** -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns:

- ESP_OK: Disable glitch filter successfully
- ESP_ERR_INVALID_ARG: Disable glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Disable glitch filter failed because the glitch filter is not enabled yet
- ESP_FAIL: Disable glitch filter failed because of other error

Structures

---

struct **gpio_pin_glitch_filter_config_t**

Configuration of GPIO pin glitch filter.

Public Members

> **glitch_filter_clock_source_t clk_src**
>
> Clock source for the glitch filter

> **gpio_num_t gpio_num**
>
> GPIO number

---

struct **gpio_flex_glitch_filter_config_t**

Configuration of GPIO flex glitch filter.

Public Members

> **glitch_filter_clock_source_t clk_src**
>
> Clock source for the glitch filter

gpio_num_t gpio_num

GPIO number

uint32_t window_width_ns

Sample window width (in ns)

uint32_t window_thres_ns

Sample window threshold (in ns), during the `window_width_ns` sample window, any pulse whose width < window_thres_ns will be discarded.

Type Definitions

typedef struct gpio_glitch_filter_t ***gpio_glitch_filter_handle_t**

Typedef of GPIO glitch filter handle.

Was this page helpful?